

shogiAI 資料庫系統與設計報告

系統架構、資料庫設計、ER 圖、Relational Model、建表 SQL、查詢、API 與 AI 訓練資料

shogiAI 專案

2026 年 6 月

摘要

shogiAI 專案的資料庫系統以 MySQL 保存 CSA 將棋棋譜，將原始棋譜轉換為可查詢、可回放、可匯出訓練資料的關聯式資料模型。系統涵蓋資料蒐集、資料匯入、ER 設計、relational model、資料表設計、CREATE TABLE 程式碼、查詢功能、HTTP JSON API、前端棋譜瀏覽、AI 訓練資料匯出、資料品質、索引與維護操作。截至 2026-06-23 21:45，遠端 MySQL 資料庫包含 80,480 筆 `game_records`、9,250,907 筆 `moves`、9,331,387 筆 `positions`、5,143 筆 `players` 與 2 筆 `users`，最晚資料寫入時間為 2026-06-23 04:56:24。

目录

1	資料規模概況	5
2	專案目標	5
3	資料庫系統目標	6
4	整體系統架構	6
5	遠端資料蒐集流程	7
6	本地與遠端同步機制	7
7	設計需求分析	8
7.1	資料本身的特性	8
7.2	系統功能需求	8
8	資料庫設計方法	8
8.1	走法與局面的核心關係	9

目录	2
9 ER 圖	9
10 Relational Model	10
10.1 users	10
10.2 players	10
10.3 game_records	10
10.4 moves	11
10.5 positions	11
11 資料表設計	11
11.1 users	11
11.2 players	12
11.3 game_records	12
11.4 moves	12
11.5 positions	13
12 如何 Create Database 與 Tables	13
12.1 手動建立資料庫	13
12.2 建立 users	14
12.3 建立 players	14
12.4 建立 game_records	14
12.5 建立 moves	15
12.6 建立 positions	15
12.7 專案中的自動建表程式	15
13 CSA 匯入 MySQL 流程	16
14 資料匯入實例	17
14.1 新增棋局	17
14.2 新增手順	17
14.3 新增局面	18
15 Position 與 Move 的對齊設計	18
16 查詢功能如何實現	19
16.1 棋局列表與搜尋	19
16.2 取得某盤棋第 n 手局面	20
16.3 資料庫統計	20
16.4 AI 訓練資料匯出查詢	21
17 後端 API 設計	21

目录	3
18 API 功能與資料庫對應	22
19 前端棋譜瀏覽器	22
20 資料庫設計如何支援各項功能	23
20.1 棋譜瀏覽	23
20.2 CSA 上傳	23
20.3 AI 訓練	24
20.4 Opening Book	24
21 資料庫如何支援 AI	24
21.1 訓練資料匯出	24
21.2 Opening Book	24
21.3 AI 對弈資料服務	25
22 資料品質與去重	25
22.1 合法手驗證	25
22.2 檔名去重	25
22.3 內容去重	25
23 資料品質與交易控制	25
23.1 合法性檢查	25
23.2 重複資料處理	25
23.3 交易控制	26
24 效能與索引建議	26
25 索引設計建議	26
26 限制與未來改進	27
27 設計限制與改進方向	28
27.1 目前限制	28
27.2 未來可改進	28
28 常用操作指令	28
28.1 匯入 CSA 到 MySQL	28
28.2 啟動瀏覽器 API	29
28.3 匯出 policy 訓練資料	29
29 系統層結論	29

1 資料規模概況

截至 2026-06-23 21:45，遠端 MySQL 資料庫的主要資料表規模如下。資料庫資料量大於 2026-06-16 模型訓練資料集，原因是爬蟲與 watcher 在訓練後仍持續匯入棋譜；模型訓練結果對應 2026-06-16 匯出的固定資料集，資料庫規模則反映目前累積狀態。

資料表或項目	截至 2026-06-23 的數值
game_records	80,480 筆
moves	9,250,907 筆
positions	9,331,387 筆
players	5,143 筆
users	2 筆
最早對局日期	1989-01-09
最晚對局日期	2026-05-19
最晚資料寫入時間	2026-06-23 04:56:24

棋局結果	game_records 筆數
TORYO	79,728 筆
SENNICHITE	751 筆
DRAW	1 筆

2 專案目標

shogiAI 的資料庫系統把原始 CSA 棋譜整理成可查詢、回放、訓練、驗證的資料資產。整體目標分成四個層次：

1. **資料蒐集**：從 shogidb2 網站取得棋局資料，轉成 CSA 檔案並保存。
2. **資料管理**：將 CSA 檔案匯入 MySQL，以正規化資料表保存棋手、對局、走法與局面。
3. **資料服務**：透過 HTTP JSON API 讓前端查詢棋局、回放任意手數、上傳新棋譜與取得資料庫統計。
4. **AI 支援**：由資料庫匯出 policy 訓練資料集，並從資料庫統計 opening book 供 AI 對弈使用。

資料庫因此是整個專案的中樞。爬蟲、前端、後端、訓練程式與 AI 搜尋系統都圍繞同一份 MySQL 資料運作。

3 資料庫系統目標

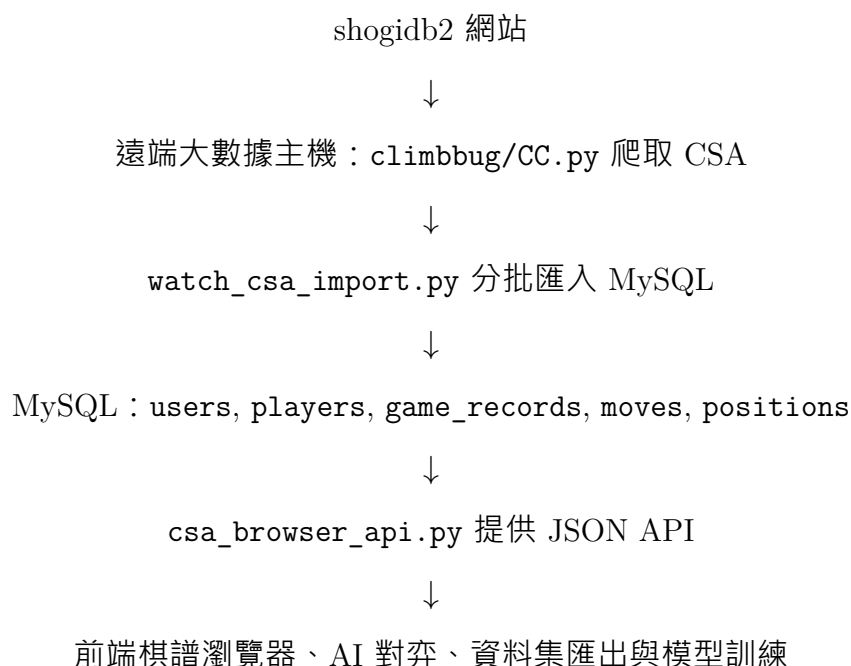
shogiAI 的資料來源是 CSA 將棋棋譜。若只把每一盤棋保存成文字檔，雖然可以保留原始資料，但無法有效支援下列需求：

- 依棋手、日期、賽事、開局或結果搜尋棋局。
- 在網頁上快速跳到任意手數並顯示棋盤。
- 上傳新的 CSA 棋譜並納入同一套資料系統。
- 匯出 AI 訓練資料，例如用目前局面預測下一手。
- 建立 opening book，統計同一局面下常見實戰手。
- 追蹤資料來源、避免重複匯入、檢查非法棋譜。

因此資料庫的核心任務不能是單純保存檔案，而要把一盤棋拆成可查詢、驗證、關聯、重複使用的資料結構。shogiAI 採用 MySQL 作為主要資料庫，並以 Python 程式負責建表、匯入、查詢與 API 回傳。

4 整體系統架構

專案由本地開發環境、遠端大數據主機、MySQL 主機與 GitHub 四個部分組成。



本地與遠端的程式碼透過 GitHub main 分支同步。遠端負責長時間蒐集資料與訓練，本地負責開發、展示、文件撰寫與模型使用。

5 遠端資料蒐集流程

遠端爬蟲位於 `climbbug/CC.py`。它的目的，是從 `shogidb2` 網站取得公開棋局並轉成 CSA 格式。流程如下：

1. 使用 `requests` 下載 `shogidb2` 的棋譜列表頁。
2. 使用 `BeautifulSoup` 解析 HTML 中的棋局連結。
3. 將棋局 URL 寫入 `game_urls.txt`，避免重複掃描。
4. 使用 `Selenium` 啟動 headless Chrome 開啟棋局頁面。
5. 從 Chrome performance log 中讀取 WebSocket frame。
6. 在 frame payload 中尋找包含 moves 與 csa 的棋局資料。
7. 將棋手、賽事、地點、時間、戰型與每一步 CSA 走法組合成完整 CSA 文字。
8. 寫入 `climbbug/data/gameN.csa`。
9. 若抓取失敗，將 URL 寫入 `failed_urls.txt`。

這個設計同時使用靜態解析與瀏覽器自動化。列表頁可用 `requests` 和 HTML parser 快速取得；棋局內容則需要 `Selenium`，因為實際資料藏在瀏覽器接收到的動態訊息中。

6 本地與遠端同步機制

專案使用 GitHub 作為共同來源。同步腳本包含：

- `sync_repo.ps1`：本地 Windows 使用，負責 `git add`、`commit`、`pull --rebase`、`push`。
- `sync_repo.sh`：Linux 環境使用，功能與 PowerShell 版本相同。
- `sync_remote.sh`：遠端大數據主機使用，會先把 live crawler 目錄 `$HOME/climbbug` 的資料複製進 repo 的 `climbbug/`，再提交推送，最後把 repo 內容同步回 live crawler 目錄。

這樣做的好處是遠端與本地不直接互相覆蓋，而是透過 Git 歷史保留每次資料與程式變更。如果兩邊改到同一檔案，Git rebase 會停下來要求解衝突，避免資料被靜默覆寫。

7 設計需求分析

7.1 資料本身的特性

一盤 CSA 棋譜同時包含多種資料：

- 棋局層級資料：賽事名稱、地點、開局、結果、對局日期、原始檔名。
- 棋手層級資料：先手棋手、後手棋手、段位或稱號。
- 手順層級資料：第幾手、行棋方、原始 CSA 走法、標準 USI 走法、註解。
- 局面層級資料：每一手前後的 SFEN、是否王手、合法手數、局面 hash。
- 系統層級資料：上傳者、匯入者、建立時間、資料來源格式。

如果把這些資料全部放在同一張大表，會發生資料重複、更新困難、查詢混亂等問題。例如同一位棋手可能出現在上千盤棋中，如果每盤棋都重複寫棋手名稱，之後要修正名稱時就必須更新很多列。因此本專案把資料拆成五張主要資料表：

users, players, game_records, moves, positions

7.2 系統功能需求

shogiAI 的主要功能與資料庫需求如下：

功能	資料庫需要支援的能力
棋譜列表與搜尋	查詢 game_records，並 join players 顯示先後手棋手。
任意手回放	以 game_id 與 move_number 直接取得 positions.sfen。
手順顯示	查詢同一 game_id 下所有 moves，依手數排序。
CSA 上傳	新增 game_records、moves、positions，並追蹤 users。
資料庫統計	計算棋局數、棋手數、走法數、局面數。
AI 訓練資料匯出	join positions 與 moves，形成 $Position_t \rightarrow Move_{t+1}$ 。
Opening book	以 sfen_hash 或 SFEN 統計相同局面的下一手分布。

8 資料庫設計方法

本專案採用關聯式資料庫 MySQL。設計時先拆出 CSA 棋譜中的主要實體：

- 使用者或匯入者。

- 棋手。
- 一盤對局的 metadata。
- 每一步走法。
- 每一手後的局面。

因此資料庫拆成五張主要資料表：`users`、`players`、`game_records`、`moves`、`positions`。這樣符合正規化精神：棋手名稱不重複存在每一步資料中，對局資料不混入局面資料，走法與局面也分開保存。

8.1 走法與局面的核心關係

本專案最重要的資料模型是：

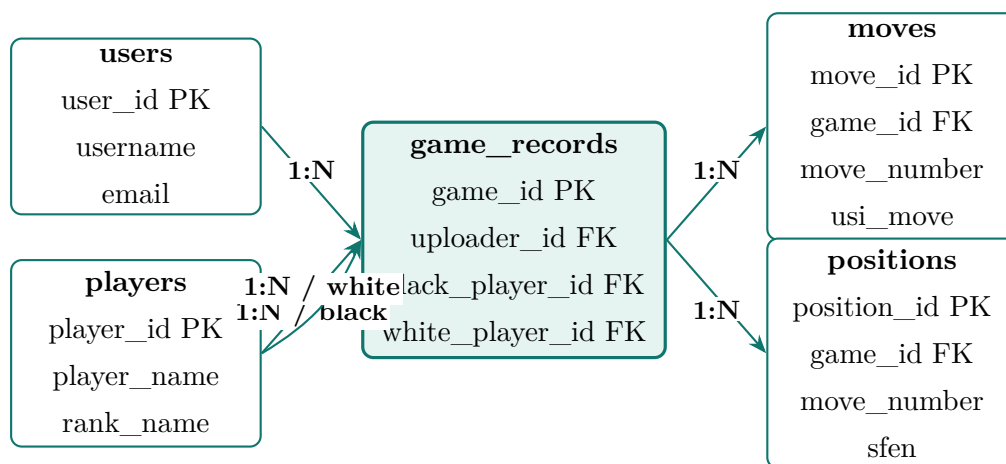
$$Position_t + Move_{t+1} \rightarrow Position_{t+1}$$

`positions` 的第 0 筆代表初始局面，第 t 筆代表走完第 t 手後的局面；`moves` 的第 $t+1$ 筆代表從 `Position_t` 出發的下一步。這個設計使系統可以：

- 直接查詢任意手數，不必每次從第一手重播。
- 從 `positions` join `moves` 匯出「局面、下一手」訓練資料。
- 用 SFEN hash 統計重複局面或建立 opening book。

9 ER 圖

本專案的 ER 圖如下。中心是 `game_records`，它代表一盤棋；一盤棋會連到兩位棋手，也會擁有多筆走法與多筆局面。若棋譜由系統匯入或使用者上傳，也會連到 `users`。



10 Relational Model

Relational model 將 ER 圖轉成可在 MySQL 建立的關聯 schema。文字中的底線表示主鍵，並列出外鍵參照。

10.1 users

```
users(user_id, username, email, password_hash, role, created_at)
```

- Primary Key: `user_id`
- Unique: `email`
- 用途：記錄棋譜上傳者或匯入者，保留資料來源責任。

10.2 players

```
players(player_id, player_name, rank_name, created_at)
```

- Primary Key: `player_id`
- 用途：保存棋手資料，避免棋手名稱在每一盤棋中重複出現。

10.3 game_records

```
game_records(game_id, uploader_id, black_player_id, white_player_id,  
             event_name, site, opening, result, source_format,  
             original_file_name, played_at, created_at)
```

- Primary Key: `game_id`
- Foreign Key: `uploader_id` references `users(user_id)`
- Foreign Key: `black_player_id` references `players(player_id)`
- Foreign Key: `white_player_id` references `players(player_id)`
- 用途：保存一盤棋的 metadata，是整個 schema 的中心表。

10.4 moves

```
moves(move_id, game_id, move_number,  
      side_to_move, original_move, usi_move,  
      move_time, comment, created_at)
```

- Primary Key: `move_id`
- Foreign Key: `game_id` references `game_records(game_id)`
- 用途：保存每一手走法，支援手順列表與棋譜回放。

10.5 positions

```
positions(position_id, game_id, move_number,  
          side_to_move, sfen, sfen_hash,  
          is_check, legal_moves_count, created_at)
```

- Primary Key: `position_id`
- Foreign Key: `game_id` references `game_records(game_id)`
- 用途：保存每一手對應局面，支援任意手查詢、AI 資料匯出與 opening book 統計。

11 資料表設計

11.1 users

`users` 保存系統使用者或匯入者，目前主要用於記錄資料來源與上傳者。

欄位	型別	用途
<code>user_id</code>	INT PK	使用者主鍵，自動遞增。
<code>username</code>	VARCHAR	使用者或匯入程式名稱。
<code>email</code>	VARCHAR UNIQUE	可選信箱，也可作為匯入者識別。
<code>password_hash</code>	VARCHAR	預留登入功能。
<code>role</code>	VARCHAR	權限角色，預設為 <code>user</code> 。
<code>created_at</code>	DATETIME	建立時間。

11.2 players

players 保存棋手資料，避免同一棋手在多盤棋中重複儲存。

欄位	型別	用途
player_id	INT PK	棋手主鍵。
player_name	VARCHAR	棋手名稱。
rank_name	VARCHAR	段位或稱號。
created_at	DATETIME	建立時間。

11.3 game_records

game_records 是對局主表，保存一盤棋的 metadata。

欄位	型別	用途
game_id	INT PK	對局主鍵。
uploader_id	INT FK	匯入者，連到 users。
black_player_id	INT FK	先手棋手，連到 players。
white_player_id	INT FK	後手棋手，連到 players。
event_name	VARCHAR	賽事名稱。
site	VARCHAR	對局地點。
opening	VARCHAR	戰型或開局名稱。
result	VARCHAR	結果，例如 TORYO、DRAW、SENNICHITE。
source_format	VARCHAR	來源格式，目前主要為 CSA。
original_file_name	VARCHAR	原始檔名，用於追溯與去重。
played_at	DATE	對局日期。

11.4 moves

moves 保存每一步棋。

欄位	型別	用途
move_id	INT PK	走法主鍵。
game_id	INT FK	所屬對局。
move_number	INT	第幾手，從 1 開始。
side_to_move	ENUM	下此手的一方，black 或 white。
original_move	VARCHAR	原始走法字串。
usi_move	VARCHAR	標準化 USI 走法。
move_time	VARCHAR	該手耗時。
comment	TEXT	棋譜註解。

11.5 positions

positions 保存局面。第 0 手是初始局面，第 n 手代表走完第 n 手後的盤面。

欄位	型別	用途
position_id	INT PK	局面主鍵。
game_id	INT FK	所屬對局。
move_number	INT	此局面對應走完第幾手。
side_to_move	ENUM	輪到哪一方行棋。
sfen	TEXT	局面的 SFEN 表示。
sfen_hash	CHAR(64)	SFEN 的 SHA-256 hash。
is_check	BOOLEAN	是否正在被王手。
legal_moves_count	INT	此局面合法手數量。

12 如何 Create Database 與 Tables

12.1 手動建立資料庫

在 MySQL 中可先建立專案資料庫。實際專案使用的資料庫名稱是 DB11211213。

```
CREATE DATABASE IF NOT EXISTS DB11211213
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;
```

```
USE DB11211213;
```

12.2 建立 users

```
CREATE TABLE IF NOT EXISTS users (  
    user_id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE,  
    password_hash VARCHAR(255),  
    role VARCHAR(50) DEFAULT 'user',  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

12.3 建立 players

```
CREATE TABLE IF NOT EXISTS players (  
    player_id INT AUTO_INCREMENT PRIMARY KEY,  
    player_name VARCHAR(100) NOT NULL,  
    rank_name VARCHAR(50),  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

12.4 建立 game_records

```
CREATE TABLE IF NOT EXISTS game_records (  
    game_id INT AUTO_INCREMENT PRIMARY KEY,  
    uploader_id INT,  
    black_player_id INT,  
    white_player_id INT,  
    event_name VARCHAR(255),  
    site VARCHAR(255),  
    opening VARCHAR(100),  
    result VARCHAR(50),  
    source_format VARCHAR(20),  
    original_file_name VARCHAR(255),  
    played_at DATE,  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (uploader_id) REFERENCES users(user_id),  
    FOREIGN KEY (black_player_id) REFERENCES players(player_id),
```

```
FOREIGN KEY (white_player_id) REFERENCES players(player_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

12.5 建立 moves

```
CREATE TABLE IF NOT EXISTS moves (
    move_id INT AUTO_INCREMENT PRIMARY KEY,
    game_id INT NOT NULL,
    move_number INT NOT NULL,
    side_to_move ENUM('black', 'white') NOT NULL,
    original_move VARCHAR(100),
    usi_move VARCHAR(20),
    move_time VARCHAR(50),
    comment TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (game_id) REFERENCES game_records(game_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

12.6 建立 positions

```
CREATE TABLE IF NOT EXISTS positions (
    position_id INT AUTO_INCREMENT PRIMARY KEY,
    game_id INT NOT NULL,
    move_number INT NOT NULL,
    side_to_move ENUM('black', 'white') NOT NULL,
    sfen TEXT NOT NULL,
    sfen_hash CHAR(64),
    is_check BOOLEAN DEFAULT FALSE,
    legal_moves_count INT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (game_id) REFERENCES game_records(game_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

12.7 專案中的自動建表程式

專案不是要求使用者手動執行每一段 SQL，而是在 `src/import_csa_to_db.py` 中定義 `CREATE_TABLES_SQL`，並以 `ensure_schema()` 自動建立資料表。

```
CREATE_TABLES_SQL = [
    """CREATE TABLE IF NOT EXISTS users (...)"",
```

```

"""CREATE TABLE IF NOT EXISTS players (...)""",
"""CREATE TABLE IF NOT EXISTS game_records (...)""",
"""CREATE TABLE IF NOT EXISTS moves (...)""",
"""CREATE TABLE IF NOT EXISTS positions (...)""",
]

def ensure_schema(cursor: Any) -> None:
    for sql in CREATE_TABLES_SQL:
        cursor.execute(sql)

```

匯入時若加上 `--create-tables` 參數，程式會先建立必要資料表：

```

python src/import_csa_to_db.py ^
--input climbug/data ^
--recursive ^
--host 140.135.65.53 ^
--port 3306 ^
--user 11211213 ^
--database DB11211213 ^
--create-tables ^
--skip-existing

```

13 CSA 匯入 MySQL 流程

匯入程式為 `src/import_csa_to_db.py`。它負責把一個 CSA 檔案或整個資料夾中的 CSA 檔案匯入 MySQL。

1. 掃描輸入路徑，支援單檔、資料夾與遞迴掃描。
2. 使用 `cslogi.CSA.Parser` 解析 CSA。
3. 解析棋手名稱、段位、賽事、地點、日期、戰型與結果。
4. 建立或取得 `users` 與 `players` 資料。
5. 建立 `game_records` 對局。
6. 用初始 SFEN 建立 `cslogi.Board`。
7. 對每一步檢查 `board.is_legal(move)`。
8. 寫入 `moves`。
9. 推進棋盤並寫入 `positions`。

10. 每盤棋獨立 commit；錯誤時 rollback，避免半盤棋進入資料庫。

匯入程式也支援 `--dry-run`、`--create-tables`、`--skip-existing`、`--max-games`、`--pause-every` 等參數，使大量資料匯入可以分批進行並避免重複。

14 資料匯入實例

14.1 新增棋局

```
cursor.execute(
    """
    INSERT INTO game_records
        (
            uploader_id, black_player_id, white_player_id,
            event_name, site, opening, result, source_format,
            original_file_name, played_at
        )
    VALUES
        (%s, %s, %s, %s, %s, %s, %s, 'CSA', %s, %s)
    """,
    (
        uploader_id,
        black_player_id,
        white_player_id,
        info.get("EVENT"),
        info.get("SITE"),
        info.get("OPENING"),
        game_result(parser),
        file_key,
        parse_played_at(info.get("START_TIME")),
    ),
)
game_id = int(cursor.lastrowid)
```

這段程式的重點是先寫入一盤棋的 metadata，取得 `game_id` 後，後面的 moves 與 positions 才能用外鍵連回這盤棋。

14.2 新增手順

```
cursor.executemany(
    """
```

```

INSERT INTO moves
    (game_id, move_number, side_to_move,
     original_move, usi_move, move_time, comment)
VALUES
    (%s, %s, %s, %s, %s, %s, %s)
    """
    move_rows,
)

```

`move_rows` 是程式解析 CSA 後產生的多筆走法資料。使用 `executemany()` 可以一次批次寫入多列，比逐列 `execute()` 更適合大量資料。

14.3 新增局面

```

cursor.executemany(
    """
    INSERT INTO positions
        (game_id, move_number, side_to_move,
         sfen, sfen_hash, is_check, legal_moves_count)
    VALUES
        (%s, %s, %s, %s, %s, %s, %s)
    """
    position_rows,
)

```

`positions` 是本專案最重要的設計之一。一般棋譜只需要手順即可重播，但如果每次都從第一手重播到第 n 手，前端跳手數與 AI 匯出資料會很慢。因此系統把每一手後的 SFEN 都存下來，讓任意手查詢變成直接查資料表。

15 Position 與 Move 的對齊設計

本專案最重要的資料庫設計是 `positions` 與 `moves` 的手數對齊。

$$Position_t + Move_{t+1} \rightarrow Position_{t+1}$$

- `positions.move_number = 0` 表示初始局面。
- `positions.move_number = n` 表示走完第 n 手後的局面。
- `moves.move_number = 1` 表示第一手。
- `moves.move_number = n` 表示第 n 手走法。

因此若要匯出 AI 訓練資料，會使用：

$$\text{positions.move_number} + 1 = \text{moves.move_number}$$

這代表「用目前局面預測下一手」。這個設計同時支援四種功能：

1. 前端棋譜回放：直接取第 n 手局面。
2. AI supervised learning：形成局面到下一手的訓練樣本。
3. opening book：統計同一局面下常見下一手。
4. 資料品質檢查：可驗證每一步走法是否能合法推進到下一局面。

16 查詢功能如何實現

16.1 棋局列表與搜尋

前端呼叫 GET /api/games 時，後端會查詢 game_records，並 join players 取得先手與後手棋手名稱。

```
SELECT
  g.game_id,
  g.original_file_name,
  COALESCE(bp.player_name, 'black') AS black_player,
  COALESCE(wp.player_name, 'white') AS white_player,
  COALESCE(g.event_name, '') AS event_name,
  COALESCE(g.opening, '') AS opening,
  COALESCE(g.result, '') AS result,
  g.played_at,
  (
    SELECT COUNT(1)
    FROM moves m
    WHERE m.game_id = g.game_id
  ) AS move_count
FROM game_records g
LEFT JOIN players bp ON g.black_player_id = bp.player_id
LEFT JOIN players wp ON g.white_player_id = wp.player_id
WHERE ...
ORDER BY g.played_at DESC, g.game_id DESC
LIMIT ? OFFSET ?;
```

搜尋條件由 build_filters() 組出，支援：

- event: 依賽事名稱模糊查詢。
- date_from: 起始日期。
- date_to: 結束日期。
- player: 先手或後手棋手名稱。
- opening: 開局名稱。

16.2 取得某盤棋第 n 手局面

前端呼叫：

```
GET /api/games/{game_id}?ply=n
```

後端會先確認手數範圍，再查詢 positions：

```
SELECT move_number, side_to_move, sfen, is_check, legal_moves_count
FROM positions
WHERE game_id = %s AND move_number = %s
LIMIT 1;
```

同時也會撈出整盤手順，讓前端可以顯示 move list：

```
SELECT move_number, side_to_move, original_move, usi_move, comment
FROM moves
WHERE game_id = %s
ORDER BY move_number;
```

這個功能之所以能有效率，是因為 positions 已經保存每個手數的 SFEN，不需要每次從頭重播棋局。

16.3 資料庫統計

GET /api/db/stats 會統計核心資料表：

```
SELECT COUNT(1) AS count_value FROM game_records;
SELECT COUNT(1) AS count_value FROM players;
SELECT COUNT(1) AS count_value FROM moves;
SELECT COUNT(1) AS count_value FROM positions;
```

前端會把這些數字顯示成資料庫狀態，讓使用者知道目前資料規模。

16.4 AI 訓練資料匯出查詢

AI 訓練資料匯出使用 `positions join moves` :

```
SELECT
  p.game_id,
  p.move_number,
  p.sfen,
  m.usi_move,
  p.side_to_move,
  g.result
FROM positions p
JOIN moves m
  ON m.game_id = p.game_id
  AND m.move_number = p.move_number + 1
JOIN game_records g
  ON g.game_id = p.game_id
ORDER BY p.game_id, p.move_number;
```

這個查詢把每一列轉成 :

`sfen → usi_move`

接著 Python 會用 `cschogi.Board(sfen)` 還原棋盤，把 `usi_move` 編碼成 `policy label`，最後存成 `.npz` 訓練資料。

17 後端 API 設計

後端位於 `src/csa_browser_api.py`，使用 Python 標準函式庫的 `ThreadingHTTPServer` 與 `BaseHTTPRequestHandler`。它同時支援 MySQL 資料來源與本機 CSA 檔案資料來源。

API	功能
GET <code>/api/games</code>	查詢對局列表，支援事件、日期、棋手、開局等篩選。
GET <code>/api/games/<id>?ply=n</code>	取得某盤棋第 n 手局面，用於棋盤回放。
GET <code>/api/db/stats</code>	回傳棋局、棋手、走法、局面與重複資料統計。
POST <code>/api/upload-csa</code>	接收前端上傳的 CSA，保存到 <code>data/uploads</code> 並匯入 MySQL。
POST <code>/api/self-play/state</code>	根據 USI 手順重播自我對弈狀態。

API	功能
POST /api/self-play/csa	將自我對弈手順輸出成 CSA 文字。
POST /api/ai-play/state	回傳 AI 對弈目前盤面與 policy 候選手。
POST /api/ai-play/move	讓 AI 搜尋並下一手。
POST /api/policy/candidates	回傳目前局面的 policy 候選手。
POST /api/model-match/step	執行模型互打一個 step。
POST /api/model-match/run	執行多局模型互打。

18 API 功能與資料庫對應

雖然本報告主軸是資料庫，但 API 是資料庫功能被前端使用的入口。下表整理每個 API 背後使用的資料表。

API	主要資料表	功能
GET /api/games	game_records, players, moves	搜尋棋局、分頁、顯示棋手與手數。
GET /api/games/<id>?ply=n	positions, moves, game_records	取得第 n 手棋盤與完整手順。
GET /api/db/stats	所有核心表	顯示棋局數、棋手數、走法數、局面數。
POST /api/upload-csa	users, game_records, moves, positions	上傳 CSA 並匯入資料庫。
POST /api/ai-play/move	positions, opening book, model	AI 對弈時取得局面、候選手與搜尋結果。

19 前端棋譜瀏覽器

前端位於 web/index.html、web/app.js 與 web/styles.css。它是一個原生 JavaScript 應用，透過 Fetch API 呼叫後端。

主要功能包含：

- **棋譜列表**：從 /api/games 載入對局，支援條件搜尋。

- **棋盤回放**：使用 `/api/games/<id>?ply=n` 跳到任意手數。
- **資料庫統計**：顯示目前 MySQL 中的資料量與重複群組。
- **CSA 上傳**：前端用 multipart form 上傳，後端匯入 MySQL。
- **自我對弈**：使用者可手動下棋並匯出 CSA。
- **AI 對弈**：使用者與 AI 對弈，畫面顯示搜尋評分、深度、節點、PV 與 policy 候選手。
- **模型互打**：選擇兩個 policy model，以相同搜尋框架互相對弈比較。

20 資料庫設計如何支援各項功能

20.1 棋譜瀏覽

棋譜瀏覽器需要三類資料：

- 棋局列表：來自 `game_records` 與 `players`。
- 棋盤局面：來自 `positions.sfen`。
- 手順列表：來自 `moves`。

因為 `positions` 保存每一手的 SFEN，前端可以在上一手、下一手、跳到最後一手之間快速切換。

20.2 CSA 上傳

上傳 CSA 後，後端會：

1. 檢查副檔名與檔案大小。
2. 保存到 `data/uploads`。
3. 呼叫匯入程式。
4. 若棋譜合法，新增 `game_records`、`moves`、`positions`。
5. 回傳新棋局的 `gameId`，前端切換到棋譜瀏覽畫面。

20.3 AI 訓練

policy-only 模型需要大量：

(目前局面, 下一手)

資料庫的 `positions` 與 `moves` 正好提供這個結構。因此 AI 訓練不需要重新讀每個 CSA 檔案，可以直接從 MySQL 取出標準化後的資料。

20.4 Opening Book

Opening book 的概念是：如果同一個局面在實戰棋譜中出現很多次，就統計高手常下哪一手。資料庫中 `sfen` 與 `sfen_hash` 讓系統能辨識相同局面，進而計算每個候選手的出現次數與比例。

21 資料庫如何支援 AI

資料庫對 AI 有三個用途。

21.1 訓練資料匯出

`src/export_policy_dataset.py` 從 MySQL 取出每個 `Position_t` 與下一手 `Move_t+1`：

```
SELECT p.game_id, p.move_number, p.sfen, m.usi_move, g.result
FROM positions p
JOIN moves m
  ON m.game_id = p.game_id
 AND m.move_number = p.move_number + 1
JOIN game_records g
  ON g.game_id = p.game_id
ORDER BY p.game_id, p.move_number;
```

匯出後會產生 `states`、`moves`、`game_ids`、`meta` 等 NumPy arrays，供 policy network 訓練。

21.2 Opening Book

`OpeningBook.from_mysql` 從 MySQL 統計前若干手中，同一 SFEN 的常見下一手。若某手出現次數達到門檻，AI 可在開局階段直接採用資料庫中的常見實戰走法。這能降低開局搜尋成本，也讓 AI 的開局更接近人類實戰資料。

21.3 AI 對弈資料服務

AI 對弈時，資料庫提供棋譜回放與訓練來源；模型提供 policy 候選手排序；傳統搜尋負責局面計算。系統已移除神經網路 value head，資料庫主要支援 policy 訓練與 opening book。

22 資料品質與去重

22.1 合法手驗證

資料品質最重要的步驟是合法性檢查。CSA 走法在寫入資料庫前會轉成 cshogi move，並以目前棋盤狀態檢查是否為合法手。若發現非法走法，該盤棋會 rollback，不會留下不完整資料。

22.2 檔名去重

`original_file_name` 可用於快速判斷同名 CSA 是否已經匯入。使用 `--skip-existing` 時，匯入程式會跳過已存在檔名。

22.3 內容去重

只靠檔名不夠，因為同一盤棋可能被重新命名。系統另外使用內容 key：

$$SHA256(\text{initial SFEN} + \text{full USI move sequence})$$

只要初始局面與完整走法序列相同，就視為同一盤棋。這能避免同一棋譜因檔名不同而重複匯入。

23 資料品質與交易控制

資料庫系統不只要能寫入資料，也要保證資料品質。

23.1 合法性檢查

匯入過程會用 cshogi 還原棋盤並檢查走法是否合法。如果某一步不合法，代表棋譜資料可能損壞或格式不正確，該盤棋不應寫入正式資料庫。

23.2 重複資料處理

系統使用兩種方式避免重複匯入：

- `original_file_name`: 檢查同一檔名是否已匯入。

- content key: 以初始 SFEN 加完整 USI 手順產生 hash，即使檔名不同也能辨認同一盤棋。

23.3 交易控制

一盤棋匯入會同時寫入多張表。如果寫到一半失敗，資料庫可能留下沒有手順的棋局或沒有局面的棋局。因此匯入程式應該以 transaction 包住整盤棋的寫入：

- 全部成功：commit。
- 任一步驟失敗：rollback。

這能保證資料庫不會留下半成品。

24 效能與索引建議

系統常見查詢包含依 game_id 與 move_number 查局面、依檔名去重、依日期或棋手搜尋對局、依 sfen_hash 統計重複局面。建議索引如下：

```
CREATE INDEX idx_game_records_original_file_name
ON game_records(original_file_name);

CREATE INDEX idx_game_records_played_at
ON game_records(played_at);

CREATE INDEX idx_moves_game_number
ON moves(game_id, move_number);

CREATE INDEX idx_positions_game_number
ON positions(game_id, move_number);

CREATE INDEX idx_positions_sfen_hash
ON positions(sfen_hash);
```

idx_positions_game_number 對前端回放最重要，因為每次跳手數都會用到。idx_moves_game_number 對資料集匯出最重要，因為匯出時需要把 position 與下一手 move 對齊。

25 索引設計建議

目前程式已能建立資料表與執行查詢。資料量已達 80,480 局與超過 933 萬筆局面資料，索引設計會直接影響前端回放、資料集匯出與 opening book 查詢速度。建議加入下列索引：

```

CREATE INDEX idx_game_records_played_at
  ON game_records (played_at, game_id);

CREATE INDEX idx_players_name
  ON players (player_name);

CREATE INDEX idx_moves_game_number
  ON moves (game_id, move_number);

CREATE INDEX idx_positions_game_number
  ON positions (game_id, move_number);

CREATE INDEX idx_positions_sfен_hash
  ON positions (sfен_hash);

```

各索引用途如下：

索引	用途
game_records(played_at, game_id)	棋局列表依日期排序與分頁。
players(player_name)	棋手搜尋與 join。
moves(game_id, move_number)	取得某盤棋全部手順。
positions(game_id, move_number)	快速定位某盤棋第 n 手局面。
positions(sfен_hash)	統計相同局面，支援 opening book。

索引能提升讀取速度，但會增加寫入成本。因此不應該對所有欄位都建立索引，而是依照實際功能的高頻查詢建立。

26 限制與未來改進

目前系統仍有一些限制：

- 匯出訓練資料時中途缺少細部進度列，資料量大時主要依程序狀態與 log 判斷進度。
- MySQL 匯入與訓練匯出可能同時發生，需要更完整的排程管理。
- 前端統計仍偏基本，可加入戰型分布、棋手分布、年份分布等分析。

- 資料庫 schema 可加入更多唯一約束與索引，以提高去重與查詢效能。
- 上傳功能目前適合內部展示，若要公開使用仍需登入、權限與檔案掃描。

27 設計限制與改進方向

27.1 目前限制

- 建表 SQL 目前主要建立基本 foreign key，尚未在程式中建立所有建議索引。
- players 尚未對棋手名稱與段位建立唯一約束，因此同名不同寫法仍可能分裂。
- positions.sfen 使用 TEXT，若大量查詢完整 SFEN，應優先查 sfen_hash。
- users.password_hash 欄位目前保留，但完整登入權限系統尚未完善。

27.2 未來可改進

- 加入 migration 工具，讓 schema 更新可以版本化。
- 補上索引建立腳本，特別是 positions(game_id, move_number)。
- 對棋手名稱建立清洗與合併規則，避免同一棋手被視為多人。
- 建立資料品質 dashboard，統計非法棋譜、重複棋譜與匯入錯誤。
- 強化 API 權限控管，限制上傳與刪除等高風險操作。

28 常用操作指令

28.1 匯入 CSA 到 MySQL

```
python src\import_csa_to_db.py ^
--input data ^
--recursive ^
--create-tables ^
--host 140.135.65.53 ^
--port 3306 ^
--user 11211213 ^
--database DB11211213 ^
--skip-existing
```

28.2 啟動瀏覽器 API

```
python src\csa_browser_api.py ^  
  --source mysql ^  
  --db-host 140.135.65.53 ^  
  --db-port 3306 ^  
  --db-user 11211213 ^  
  --db-name DB11211213 ^  
  --policy-model models\policy_model.pt ^  
  --policy-order-ply 2 ^  
  --opening-book-ply 30 ^  
  --opening-book-min-count 2
```

28.3 匯出 policy 訓練資料

```
python src\export_policy_dataset.py ^  
  --output out\policy_dataset.npz ^  
  --host 140.135.65.53 ^  
  --port 3306 ^  
  --user 11211213 ^  
  --database DB11211213
```

29 系統層結論

shogiAI 的資料庫系統將 CSA 棋譜轉換成可查詢、回放、訓練的關聯式資料。遠端爬蟲負責持續蒐集棋譜，後端 API 負責把資料轉成前端與 AI 可用的 JSON，前端提供使用者互動介面是串接資料蒐集、棋譜管理、AI 訓練與 AI 對弈的核心基礎。截至 2026-06-23，資料庫已累積 80,480 局與超過 925 萬筆走法資料，足以支援棋譜瀏覽、資料統計、opening book 與後續模型再訓練。

A 資料庫設計結論

shogiAI 的資料庫設計將 CSA 棋譜轉成可查詢、回放、訓練的資料模型。五張核心表各自負責不同概念：

- users: 資料來源與上傳者。
- players: 棋手資料。
- game_records: 棋局 metadata。

- moves: 每一步走法。
- positions: 每一手對應局面。

其中 positions 與 moves 的對齊設計是整個系統的核心，因為它同時支援前端任意手回放、AI 訓練資料匯出與 opening book 統計。